IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

# Yenepoya Institute of Arts , Science Commerce & Management

## Project – Low Level Design

## on

## NATURAL LANGUAGE INTERFACE FOR E-COMMERCE INFRASTRUCUTRE

**Student Name:** ARYAN VIMOD

**Industry Mentor:Mr.Sumit Kumar**

**Guided by: Mr.Praveen**

# Table of Contents


| | |
|---|---|
| 1. Introduction | Pg no:3 |
| 2. Project Goal | Pg no:3 |
| 3. Model Development | Pg no:4 |
| 4. Process Flow Diagram | Pg no:4 |
| 5. Working | Pg no:5 |
| 6. Data Design | Pg no:5 |
| 7. Challenges | Pg no:6 |
| 8. References | Pg no:6 |

# INTRODUCTION

With the rapid growth of E-commerce platforms, customer interaction and support have become essential components of user experience. Chatbots are increasingly used to automate responses and provide instant assistance.

This project focuses on **developing a Command Line Interface (CLI)-based chatbot** using Python that can handle basic E-commerce-related queries through simple string matching techniques.The chatbot is designed to simulate basic customer support by responding to user queries using simple string matching and conditional logic.

The system is lightweight, easy to implement, and runs entirely on a command-line interface without requiring external dependencies or databases.

# PROJECT GOAL

The main objectives of this project are:

- To develop a simple chatbot using Python
- To simulate basic customer support for an E-commerce system
- To understand string processing and conditional logic (`if/else`)
- To build a lightweight, interactive CLI tool
- Covers internal design and implementation details
- Includes modules, functions, logic flow, and data handling
- Focuses on CLI interaction only (no GUI/web)

# MODEL DEVELOPMENT

The chatbot model is developed using a rule-based approach:

- Uses **keyword detection** instead of NLP
- Implements **if-else conditions** for decision making
- Stores responses in **dictionary format**

### Core Logic:

- Convert user input to lowercase
- Identify keywords like *status*, *return*, *help*
- Map keywords to predefined responses

# PROCESS FLOW DIAGRAM

Start

↓

User Input

↓

Convert to Lowercase

↓

Keyword Matching

↓

Response Generation

↓

Display Output

↓

Repeat / Exit

# WORKING

1. The chatbot starts and greets the user
2. User enters a query in the CLI
3. Input is preprocessed (lowercase, trimmed)
4. System checks for keywords
5. Appropriate response is selected
6. Output is displayed
7. Loop continues until user types “exit”

The chatbot operates in a continuous loop where it accepts user input through the command line interface. The input is preprocessed by converting it to lowercase and removing unnecessary spaces.

The system then checks for specific keywords within the input and maps them to predefined responses. Based on the matched keyword, an appropriate response is displayed to the user.

This process continues until the user enters an exit command, ensuring smooth and interactive communication.

# DATA DESIGN

The chatbot uses an in-memory dictionary:

```
responses = {
   "status": "Your order is being processed.",
   "return": "You can return products within 7 days.",
   "help": "Ask about orders, returns, or products."
}
```

## Data Handling:

- No database used
- Data is static and predefine

- Accessed using direct lookup or condition checks

## CHALLENGES

- Limited understanding of user input
- Cannot process complex or natural language queries
- Requires exact or similar keywords
- No learning or adaptive capability
- Scalability is limited without NLP integration

## REFERENCES

- GitHub
- Google Chrome
- ChatGPT